

Hierarchical ensemble-based feature selection for time series prediction[☆]

Huseyin Karaca^{ID,*}, Aysin Tumay¹, Ali Taha Koc, Suleyman Serdar Kozat

Bilkent University, Ankara, 06800, Türkiye

ARTICLE INFO

Keywords:

Feature selection
Ensemble learning
The curse of dimensionality
Hierarchical ensemble
Light gradient boosting machine (LightGBM)
Time-domain signal prediction

ABSTRACT

We introduce a hierarchical ensemble-based feature selection (HEFS) method specifically designed for high-dimensional, non-stationary time series prediction tasks. Traditional methods typically select a single dominant subset of features, potentially neglecting valuable information from non-selected subsets. In contrast, our approach employs a multi-layer hierarchical structure, where dominant features are initially used to train the first layer, and non-dominant features are sequentially processed in subsequent layers. A novel cost-optimization block bridges each layer, recalibrating the predictions of the previous layer by directly minimizing the final loss. Thus, effectively capturing interactions across feature groups in different layers and overcoming some of the limitations of classical gradient boosting-based ensembles. Extensive experiments on synthetic and real-world datasets demonstrate that our method significantly outperforms classical approaches in predictive accuracy and robustness against overfitting. To promote transparency and reproducibility, we publicly share our implementation code.

1. Introduction

1.1. Preliminaries

We study feature selection for time-domain signal prediction tasks under scenarios where the number of features is large relative to the number of available samples. This challenge is widely recognized in signal processing literature due to the so-called “curse of dimensionality,” which implies that estimation models can struggle when the sample size is insufficient compared to the feature count [1,2]. Consequently, the system may become over-parameterized (overfitted), resulting in high variance and weak generalization [3,4].

Importantly, in high-dimensional regimes, overfitting is not limited to individual data points, but can also occur with respect to the feature set itself: learning algorithms tend to overfit to a small subset of dominant features, often those that are highly correlated with the target, while systematically underutilizing or even ignoring the remaining informative features [5]. This feature selection problem is even more prominent in non-stationary settings, e.g., time-varying or drifting signals, where trends and relationships between features and the target output change significantly over time, making it challenging to identify relevant components.

Traditionally, the overfitting challenge in high-dimensional scenarios has been addressed by reducing the number of features through various selection strategies: i) using statistical measures to filter features (filters, [6]), ii) exhaustively considering all feature subsets (wrappers, [7]), and iii) embedding feature selection within the learning or estimation process itself (e.g., Lasso [8], Random Forest [9]). These approaches have proven highly effective in many settings, each offering particular advantages depending on the problem structure, data characteristics, and computational constraints.

However, a common limitation across these methods is that—despite their strengths—they typically focus on identifying and leveraging only a single subset of features deemed most relevant, often leaving aside other potentially informative features. In scenarios where informative signals are distributed across both dominant and non-dominant features, this can result in some relevant information being underutilized. Moreover, for time-varying or non-stationary signals, the set of most informative features may itself change over time, further complicating the selection process.

Here, we introduce a hierarchical boosting-based ensemble approach for feature selection, in which we first train a model on a dominant subset of features and then sequentially pass the residual feature sets through additional layers. The assignment of features to layers is inher-

[☆] This work was supported by the Scientific and Technological Research Council of Turkey and the Turkish Academy of Sciences Outstanding Researcher Program.

* Corresponding author.

E-mail addresses: huseyin.karaca@bilkent.edu.tr (H. Karaca), aysintumay@gmail.com (A. Tumay), atkoc@ee.bilkent.edu.tr (A.T. Koc), kozat@ee.bilkent.edu.tr (S.S. Kozat).

¹ Author was affiliated with Bilkent University during the conduct of this research and is now at UC San Diego, CA 92093, United States.

ently flexible: the dominant subset for the first layer can be determined using any relevance measure—such as univariate or multivariate scoring functions, classical information criteria (AIC [10], BIC [11], SIC [12], UAED [13]), multi-criteria decision making systems [14], or even direct domain expertise. This allows the practitioner to tailor the grouping of features according to the specifics of the data or application. As a result, our framework is agnostic to the initial feature selection strategy, and feature groups can be formed using a single metric, a combination of multiple criteria, or manually defined by the user.

At each stage of the hierarchy, the model processes a different subset of features, ensuring that both dominant and non-dominant feature groups are explicitly utilized in the prediction process. This hierarchical structure allows information that would be ignored or discarded in single-stage selection approaches to be included through subsequent layers. Between each stage, a cost-optimization block is applied. Unlike classical residual boosting, where the next stage operates on the raw residuals or gradients, our method recalibrates the predictions by searching for a greedy scaling factor that directly minimizes the overall loss. This adjustment not only increases flexibility for different loss functions, but also enables the model to exploit dependencies and interactions between different feature groups across layers, addressing a key limitation of standard boosting algorithms (see Section 3 and [15]).

It is important to clarify that our approach does not fall under the classic filter, wrapper, or embedded feature selection categories. Our method does not rely on statistical criteria (filter), does not search by training different models on various feature subsets (wrapper), and does not perform end-to-end feature selection jointly with model training (embedded). Instead, our approach follows a hierarchical, boosting-based ensemble scheme, similar in spirit to ensemble models as gradient boosting machines, where different feature groups are processed in a layered, coordinated manner. This ensemble-based structure allows for effective use of all features by mitigating the dominant effects of certain features and making use of domain knowledge in grouping.

1.2. Related work

Feature selection has been extensively studied in the signal processing and machine learning literature. Classical techniques include filter methods, which score features using statistical criteria such as correlation, mutual information, or chi-square tests [6,16–18], and wrapper methods, which evaluate the predictive value of feature subsets through forward or backward selection using a model-based objective [7,19]. Information-theoretic criteria—most notably Akaike’s Information Criterion (AIC) and the Bayesian Information Criterion (BIC)—are also routinely employed to control model dimensionality in a filter-like fashion [10,11]. Recent studies have proposed elbow-detection techniques such as UAED and SIC to efficiently estimate the effective number of features by leveraging error-curve geometry, thereby reducing the computational burden of wrapper-based selection [12,13].

While filters are computationally efficient and model-independent, they often overlook dependencies between features, treating each variable in isolation. Wrappers, by contrast, can exploit correlations among features, but their exhaustive search over subsets becomes intractable in high-dimensional settings; thus, heuristic or greedy methods are typically used, sometimes at the cost of missing relevant interactions. Mixed heuristics such as the Gibbs-guided variable selection combine ranking filters with probabilistic exploration, offering improved results for high-dimensional models [20].

Embedded methods, such as Lasso regression [8] or tree-based models such as Random Forests [9], incorporate feature selection within the learning process itself. These methods assign feature importances—often using univariate impurity or gain metrics—and have become widely adopted due to their simplicity and interpretability. However, embedded methods also have limitations: they typically score each feature independently and may not fully capture cross-correlations among features. Some recent works address these issues with joint optimiza-

tion frameworks [21], but many such methods require differentiable loss functions and are often task-specific.

A growing body of research investigates ensemble-based and deep learning-based feature selection. Hybrid frameworks, such as wrapper-filter ensembles, combine the speed of filters with the accuracy of wrappers by using filter methods to narrow down candidate features before wrapper-based evaluation [22,23]. Boosting-based methods, including those based on decision stumps, iteratively reweight feature subsets to maximize information gain [24]. Recent deep-learning approaches exploit neural architectures (e.g., LSTM or GCN) to track complex feature interactions and adapt to non-stationarity, as in reinforcement learning wrappers or attention-enhanced temporal GCNs [25,26]. However, these approaches often lack a principled way to incorporate practitioner domain knowledge or to exploit multi-criteria feature selection across multiple hierarchical stages.

In addition to the pitfalls of univariate scoring, relying on a single selection criterion—such as keeping only the features favored by a specific statistic (e.g., AIC) or by domain intuition alone—may also deteriorate overall model performance. Recent studies in skeleton-based gait analysis exemplify this risk: the Siamese denoising auto-encoder in [27] removes noisy joints and re-injects cleaned trajectories, while the attention-enhanced temporal GCN in [26] learns joint subsets optimized for different tasks. Both works observe that when feature selection is driven by a single objective, the chosen feature set can dominate the learning signal and cause more subtle, yet informative, features to be overlooked or suppressed.

Finally, there is a growing interest in combining domain knowledge with statistical learning for feature selection. Recent work by Jenul et al. [28] demonstrates a user-guided ensemble approach that integrates expert priors and model likelihood in the selection process. Mixed-heuristic and multi-criteria decision-making strategies (e.g., MCDM) offer further flexibility but often lack mechanisms for hierarchical exploitation of both dominant and non-dominant feature groups.

Overall, most existing methods select a single subset of features and use them throughout the learning process, which may result in overlooked, yet informative, signals—especially in non-stationary or high-dimensional settings. In contrast, our approach explicitly addresses these limitations by hierarchically layering feature groups, flexibly integrating domain expertise, and fully exploiting cross-group dependencies in both dominant and residual subsets.

1.3. Contributions and organization

Our chief contributions are as follows:

1. We introduce a flexible framework that hierarchically assigns both dominant and non-dominant features—identified through any feature selection method or domain expertise—across multiple layers. This layered approach enables the effective exploitation of information that would otherwise be overlooked by traditional single-pass feature selection methods.
2. To connect predictions from models trained on different feature subsets, we introduce a cost-optimization block that adaptively refines layer outputs—even for objective functions lacking practical second derivatives (e.g., L1 loss). This circumvents the limitations of classical residual boosting, where base learners for non-differentiable losses (such as mean absolute error) are forced to predict only ± 1 values, leading to suboptimal performance [3,15]. Our adjustment strategy enables robust ensemble learning for a much broader class of loss functions.
3. By explicitly optimizing layer-wise adjustment coefficients in the cost-optimization block, our method can model multivariate dependencies across feature groups. This allows the model to capture complex cross-correlations between feature sets—an ability missing in classical stacking-based ensemble techniques.

4. We provide an open-source implementation of our hierarchical ensemble framework for reproducibility and future research.²

The remainder of the paper is organized as follows: Section 2 formally introduces the problem and outlines existing feature selection paradigms. Section 3 details our hierarchical ensemble-based model, including layer construction and the cost-optimization block. Section 4 presents comprehensive experiments on both synthetic and real-world datasets, benchmarking our approach against traditional and state-of-the-art alternatives. Finally, Section 6 summarizes our findings and discusses limitations with future research directions.

2. Problem description

All vectors in this paper are column vectors in lowercase boldface type. Matrices are denoted by uppercase boldface letters. Specifically, $\mathbf{X}$ represents a matrix containing $\mathbf{x}_t^{(k)}$, i.e., the k^{th} sequence of vector $\mathbf{x}_t$, and $\mathbf{x}_t^{(k)}$ in the k^{th} column for each time t . $X_{i,j}$ denotes the element of $\mathbf{X}$ in the i^{th} row and j^{th} column. Ordinary transposes of $\mathbf{x}_t$ and $\mathbf{x}$ are denoted as $\mathbf{x}_t^\top$ and $\mathbf{x}^\top$, respectively. The mean and standard deviation of $\mathbf{x}_t^{(k)}$, i.e., the k^{th} dimension of $\mathbf{x}_t$, are denoted by $\bar{x}_t^{(k)}$ and $\sigma(\mathbf{x}_t^{(k)})$, respectively. The gamma function, having the property of $\Gamma(N) = (N-1)!$ generalizes the concept of a factorial to real and complex numbers.

We study feature selection in time series prediction of a sequence $\{y_t\}$. We observe this target sequence along with a side information sequence (or feature vectors) $\mathbf{x}_t$ each of which is of size M . At each time t , given the past information $\{y_k\}$, and $\mathbf{x}_k$ for $k \leq t$, we produce the output $\hat{y}_{t+1}$. Hence, in this setting, our goal is to find the relationship

$$\hat{y}_{t+1} = F_t(\{y_t, y_{t-1}, \dots, y_1\}, \{\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_1\}),$$

where F_t is an unknown function of time, which models $\hat{y}_{t+1}$. We introduce a hierarchical nonlinear ensemble model for F_t with an efficient feature placement procedure integrated in.

Formally, at time step t , we have a matrix $\mathbf{X}$ with a length N and dimensionality M , i.e., the N data points of $\mathbf{X}$ form an Euclidean space of M -dimensions. Considering this hypersphere, we generalize the distance between data points in the M -dimensional hypersphere as $d \approx 2 \cdot r$, where r is the radius of the hypersphere. If M approaches N , the volume of the hypersphere, $V_M(r)$, given by:

$$V_M(r) = \frac{\pi^{\frac{M}{2}}}{\Gamma(\frac{M}{2} + 1)} r^M \quad (1)$$

increases exponentially, leading to a sparser data distribution characterized by a larger d [29,30].

We also encounter data sparsity caused by non-stationarity, which typically arises from trends and seasonality, when performing time-domain forecasting for a target signal $\{y_t\}$ [31]. Such trends and seasonality reduce the effective number of distinct observations, often leading to over-parameterization (overfitting). Consequently, datasets with relatively few samples and features (of comparable number to the samples) can be prone to poor generalization [32].

To this end, we introduce a hierarchical ensemble-based feature selection method for the time-series signal forecasting task to address overfitting and non-stationarity. We partition the full feature vector $\mathbf{x}_t$ into K non-overlapping subsets $\mathbf{s}_t^{(k)}$, $k = 1, \dots, K$, where each subset can be determined either via domain knowledge or using data-driven heuristics such as feature importance scores from baseline models, or information-theoretic criteria such as AIC [10], BIC [11], SIC [12], or UAED [13], or any other feature selection - importance scoring technique.

After splitting, we train K signal estimators in a sequential, hierarchical manner: each model receives as input a particular subset $\mathbf{s}_t^{(k)}$ and is optimized according to the objective function, conditioned on the outputs from previous layers. The hierarchical structure is defined by the order in which the feature subsets $\mathbf{s}_t^{(k)}$ are presented to each stage. In our experiments, dominant features are often selected using univariate importance metrics (e.g., lagged values of the target series), but the framework is fully flexible and alternative strategies can be used to identify the dominant subset at the initial stage. Non-dominant features—such as side information, date, or calendar features—are then allocated to subsequent layers, ensuring that all relevant information is progressively utilized throughout the hierarchy.

Before describing our approach, we discuss the well-known approaches to this problem for completeness in the following.

2.1. Common approaches

A variety of strategies have been developed to address the curse of dimensionality in feature selection. The most widely adopted methods in signal processing and machine learning can be grouped into filter-based, wrapper-based, and embedded approaches, along with ensemble methods that combine several of these paradigms.

Throughout this section, let $\mathbf{x}_t \in \mathbb{R}^M$ denote the full feature vector at time (or index) t , and let $\mathbf{s}_t^{(k)} \subseteq \mathbf{x}_t$ represent a generic subset of features, where k indexes all possible feature subsets.

2.1.1. Filtering methods

Filtering methods are among the simplest and fastest approaches for feature selection. In this paradigm, each feature is scored independently according to a statistical criterion—such as the Pearson correlation coefficient [17], mutual information [16], or a statistical test—and features with the highest scores are retained. For example, the Pearson correlation coefficient evaluates the linear association between a feature and the target variable, computed as

$$S(\mathbf{x}^{(k)}) = \frac{\text{cov}(\mathbf{x}^{(k)}, \mathbf{y})}{\sigma(\mathbf{x}^{(k)}) \cdot \sigma(\mathbf{y})} = \frac{\sum_{i=1}^N (x_i^{(k)} - \mu_{\mathbf{x}^{(k)}})(y_i - \mu_y)}{\sqrt{\sum_{i=1}^N (x_i^{(k)} - \mu_{\mathbf{x}^{(k)}})^2 \sum_{i=1}^N (y_i - \mu_y)^2}},$$

where N is the number of data points, $x_i^{(k)}$ denotes the i th sample of the k th feature, y_i is the i th target value, and $\mu_{\mathbf{x}^{(k)}}$, μ_y are sample means.

Filtering methods are highly valued for their computational efficiency and for being agnostic to the downstream model; ranking is performed without fitting a model to the data. However, their main limitation is that features are evaluated independently, without regard to their interactions or combined effect. This univariate perspective may overlook sets of features that are weak individually but informative when used together. In addition, filtering methods generally do not leverage domain knowledge, relying solely on statistical associations in the data.

2.1.2. Wrapper-based methods

Wrapper-based methods approach feature selection as a model-based search problem: rather than evaluating features individually, they assess subsets of features by actually fitting and validating a predictive model for each candidate subset. In contrast to filter methods—which are inherently univariate—wrappers are multivariate: they directly account for interactions and joint contributions between features. The core idea is to select the subset that yields the best model performance, typically as measured on a held-out validation set.

Mathematically, given the full feature vector $\mathbf{x}_t \in \mathbb{R}^M$, we denote a candidate subset by $\mathbf{s}_t^{(k)} \subseteq \mathbf{x}_t$. The wrapper seeks the optimal subset by minimizing a validation loss:

² <https://github.com/huseyin-karaca/hierarchical-feature-selection>.

$$s_t^* = \arg \min_{s_t^{(k)} \subset x_t} L_{val} = \arg \min_{s_t^{(k)} \subset x_t} \sum_{t'=t_1+1}^{t_2} \ell(y_{t'}, f_w(y_{t'}, s_{t'}^{(k)})), \quad (2)$$

where f_w is a predictive model with parameters w , trained on the selected feature subset over the training window $t = 1, \dots, t_1$, $\ell(\cdot, \cdot)$ is a loss function (e.g., squared error), and the sum is taken over a validation window $t' = t_1 + 1, \dots, t_2$.

Wrapper methods are attractive because they directly optimize for predictive performance, naturally considering feature dependencies. However, their main drawback is computational complexity: exhaustively evaluating all 2^M possible subsets quickly becomes infeasible as the number of features grows [7]. Heuristic search algorithms—such as forward or backward selection—are often used to reduce computation, though these may still miss important interacting feature sets [33].

2.1.3. Embedded methods

Embedded methods perform feature selection as an inherent part of the model training process. Rather than treating selection as a separate preprocessing step (as in filters and wrappers), these methods integrate feature selection directly into the model's optimization objective. Well-known examples include Lasso regression—which imposes an ℓ_1 penalty to drive some coefficients to zero—and tree-based algorithms such as Random Forests and Gradient Boosting Machines, where feature importance scores are derived from the model's internal structure.

The main strength of embedded methods is their efficiency: selection and model fitting occur simultaneously, often yielding a good trade-off between predictive accuracy and computational cost. They are also naturally suited to high-dimensional data and can adaptively identify irrelevant or redundant features during training.

However, most embedded methods rely on univariate importance measures—for example, the total impurity reduction attributed to each feature in decision tree ensembles—which means that inter-feature correlations and joint effects may not be fully captured. Some advanced variants address this via iterative selection or joint optimization, but these approaches can be model-specific and may require differentiable loss functions.

2.1.4. Ensemble-based methods

Ensemble-based methods combine the predictive strengths of multiple models (base learners) to achieve improved performance, robustness, or generalization compared to any single model alone. In the most general form, each base learner can be of a different type (e.g., tree, linear, neural net) and may use the full feature set or any subset of features. This flexibility allows ensembles to capture diverse patterns and mitigate model-specific biases.

In classical ensemble designs, each base model is typically trained independently—often on the entire feature set—which can lead to increased computational cost, especially for large or complex base learners. Moreover, when all features are always used together, there is a persistent risk that highly dominant features will overshadow or suppress the contribution of weaker but potentially informative features. Thus, standard ensembles may still fail to exploit the full predictive value hidden in non-dominant feature groups. Hence, in our experiments, we adopt a more targeted strategy: we use the *same base model architecture* for each learner, but train each one on a distinct (and possibly non-overlapping) subset of features, such as dominant and non-dominant groups.

Formally, let K base learners be trained, each on a chosen subset of the feature vector x_t . The ensemble output is computed as a weighted combination:

$$\tilde{y}_t^E = \alpha_t^\top \tilde{y}_t,$$

where $\tilde{y}_t = [\tilde{y}_t^{(1)}, \dots, \tilde{y}_t^{(K)}]^\top$, and α_t is a vector of combination weights. The optimal weights are found by minimizing the ensemble loss, often with the constraint $\alpha_t^\top \mathbf{1} = 1$:

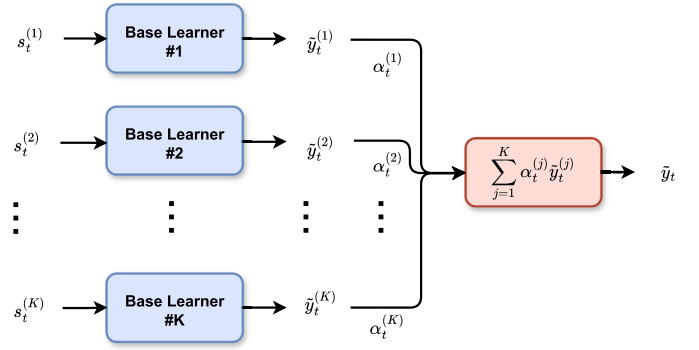


Fig. 1. We have K subsets of features (blue), each fed to a different base model. The models' predictions are then combined (pink) using the weight vector $\alpha_t^{(i)}$ for the final output.

Algorithm 1 Ensemble Model.

```

 $\tilde{y}_t^{(1)} = g(s_t^{(1)}, y_t)$  ▷ Train base learner  $g$  on subset  $s_t^{(1)}$ .
 $\tilde{y}_t^{(2)} = h(s_t^{(2)}, y_t)$  ▷ Train base learner  $h$  on subset  $s_t^{(2)}$ .
for  $t = 1$  to  $N$  do
   $\text{min\_loss} \leftarrow \infty$ 
   $\alpha_{\text{best}} \leftarrow 0$ 
  for  $\alpha = 0$  to  $1$  (step size assumed) do
     $\tilde{y}_t = \alpha \tilde{y}_t^{(1)} + (1 - \alpha) \tilde{y}_t^{(2)}$ 
     $\text{loss} = \ell(y_t, \tilde{y}_t)$ 
    if  $\text{loss} < \text{min\_loss}$  then
       $\text{min\_loss} \leftarrow \text{loss}$ 
       $\alpha_{\text{best}} \leftarrow \alpha$ 
    end if
  end for
end for

```

$$\min_{\alpha_t} \ell(y_t, \sum_{i=1}^K \alpha_t^{(i)} \tilde{y}_t^{(i)}), \quad \text{subject to } \alpha_t^\top \mathbf{1} = 1.$$

Fig. 1 presents the architecture of the algorithm for arbitrary K , and Algorithm 1 provides the pseudocode for $K = 2$.

While ensemble methods can be powerful, they are often independently trained on subsets, making it hard to exploit cross-correlation among feature groups. Furthermore, they can be computationally expensive if each subset and combination requires large-scale optimization. However, with the help of a cost-optimization block, our ensemble-based method aims to address (i) the problem of under-exploiting cross-correlation between feature groups, and (ii) the increasing computational burden of training multiple models with the all features repeatedly.

Our method can be considered in this category, aiming to solve (i) the increasing computational burden of training multiple models with the full set of features, and (ii) the lack of exploitation of cross-correlation between feature groups when base learners are not trained with the full subset.

3. Hierarchical ensemble-based approach

Our primary goal is to overcome the limitations of standard feature selection techniques by introducing a hierarchical ensemble of K machine learning models, each responsible for a different layer in a multi-stage architecture. Fig. 2 demonstrates a two-layer example. Unlike standard boosting—where only residuals are passed—each model in our hierarchy refines and scales the predictions of the previous model, using a cost-optimization block. Every model in the architecture operates on a (potentially distinct) subset of features, thus reducing overfitting and enabling the discovery of diverse, complementary patterns in the data-generating process.

Importantly, our approach is not a classical feature selection algorithm; rather, it is designed to utilize both dominant features (as would

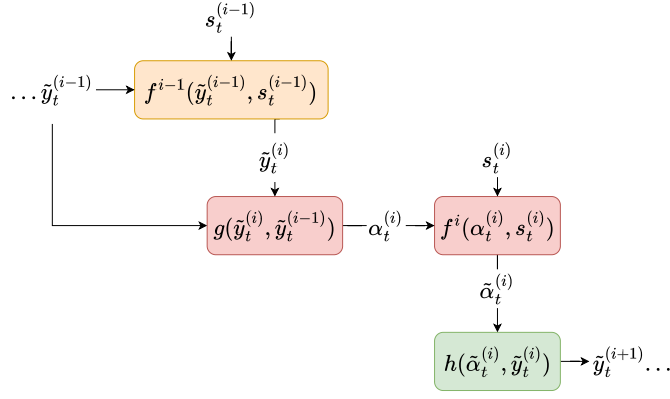


Fig. 2. Example of two-layer hierarchical modeling. The first layer (orange) uses y_t -related features as input. A cost-optimization loop computes $\alpha_t^{(i)}$. The second layer (pink) learns to predict $\alpha_t^{(i)}$ from additional exogenous signals. Refined predictions (green) combine $\tilde{\alpha}_t^{(i)}$ and $\tilde{y}_t^{(i)}$. Here, $g(\cdot, \cdot)$ and $h(\cdot, \cdot)$ are the gradient and hessian of the loss function $\ell(\cdot, \cdot)$, as in Eq. (5) and Eq. (6).

be selected by conventional methods) and non-dominant or “residual” features through a multi-layer structure. The layers are connected via a cost-optimization block, which ensures that each stage’s output contributes directly to the final loss, allowing both dominant and non-dominant feature groups to be jointly leveraged. The overall workflow is as follows:

1. Dominant features are assigned to the first layer. The selection of these features can be via any method—univariate or multivariate, single-criterion or multi-criterion, or domain-driven. In our experiments, we used univariate scoring for simplicity, but the design is flexible.
2. The cost-optimization block computes scaling coefficients (alphas) for each data point after each layer. These coefficients serve as regression targets for the next layer, facilitating the capture of complex, cross-group feature interactions.
3. Intra-group dependencies (within a layer) are modeled by the base learner (e.g., decision trees), while inter-group dependencies (across layers) are exploited via the cost-optimization block. This architecture enables the effective use of both within- and between-group information, surpassing the capabilities of classical univariate selection and standard ensemble methods.

Let y_t denote the target series and x_t the set of all exogenous (side) signals, and let $s_t(y)$ be the set of features derived solely from y_t (such as lags and rolling statistics). We generally assign $s_t(y)$ to the first layer because these features often receive higher importance scores than side-information signals and tend to dominate prediction [5]. However, our algorithm is fully flexible: feature groups for each layer can also be determined by alternative criteria, such as classical information metrics (AIC, BIC [10,11]), their modern variants (SIC, UAED [12,13]), or multi-criteria decision-making methods [14]. The main contribution of our approach is not in further optimizing dominant features, but in effectively utilizing non-dominant features through additional hierarchical layers. For example, in some experiments, we sorted features by a scoring metric (such as mutual information [16]) and used hyperparameter tuning to decide how many top-scoring features to place in the first layer. This demonstrates that our algorithm supports both domain expertise and data-driven feature grouping, allowing for flexible assignment based on the application.

3.1. Description of the model components

Each pair of layers in the hierarchy comprises four components: (i) a base model $f^{(i-1)}$ using a selected feature group, (ii) a cost-optimization

function for scaling previous predictions, (iii) a new model $f^{(i)}$ that predicts the scaling factor based on new features, (iv) a linear function h to merge scaling with earlier predictions.

3.1.1. Initial model

At layer $(i-1)$, model $f^{(i-1)}$ is trained on the feature subset $s_t^{(i-1)}$ (commonly the target-based features in the first layer), yielding predictions $\hat{y}_t^{(i-1)}$. The model and its loss function are not constrained and may be any standard method.

3.1.2. Cost optimization block

Rather than passing $\hat{y}_t^{(i-1)}$ forward unchanged, we introduce a scaling factor $\alpha_t^{(i)}$ per time t to recalibrate predictions under the final loss $\ell(\cdot, \cdot)$. Specifically,

$$L_t = \ell(y_t, \hat{y}_t^{(i)}) = \ell(y_t, \alpha_t^{(i)} \hat{y}_t^{(i)}) \quad (3)$$

where $\hat{y}_t^{(i)}$ is the scaled prediction. We search for $\alpha_t^{(i)}$ within a tunable range $[1 - \beta, 1 + \beta]$ for each t to minimize $\ell(y_t, \hat{y}_t^{(i)})$. The optimization objective is thus

$$\min_{\alpha_t \in [1 - \beta, 1 + \beta]} \ell(y_t, \alpha_t^{(i)} \hat{y}_t^{(i)}), \quad (4)$$

where ℓ can be any loss. Here, $\beta > 0$ is a hyperparameter controlling the feasible search interval for $\alpha_t^{(i)}$, always restricted to $[0, 2]$, so as to allow both attenuation ($\alpha < 1$) and moderate amplification ($\alpha > 1$), but prevent instability. This design also provides flexibility to handle arbitrary loss functions (including L_1), which is discussed in the following remark:

Remark. In classical gradient boosting frameworks, each base (weak) learner is trained to correct the shortcomings of the current ensemble by fitting to the negative gradient of the loss with respect to the model’s current prediction [34]. When the final loss is smooth and differentiable, such as the squared error (L_2), this produces continuous-valued gradients, allowing the next base learner to capture nuanced corrections and achieve stable improvement. However, for non-differentiable losses as L_1 , the gradient becomes discrete—taking values of $+1$ or -1 —which in practice forces base learners into binary regression. While theoretically valid, this can cause degradation in learning performance and stability [15].

This limitation persists in advanced implementations as LightGBM, which is a fast, histogram-based implementation of gradient-boosted decision trees, and rely on both gradient and hessian computations to construct tree splits and optimize the ensemble [35]. For a general custom loss ℓ , the gradient and hessian are defined as:

$$g_t(x) = E_y \left[\frac{\partial \ell(y_t, f(x_t, \theta, y_t))}{\partial f(x_t, \theta, y_t)} \mid x \right]_{f(x_t, \theta, y_t) = \hat{f}_{t-1}(x_t, \theta, y_t)}, \quad (5)$$

$$h_t(x) = E_y \left[\frac{\partial^2 \ell(y_t, f(x_t, \theta, y_t))}{\partial f(x_t, \theta, y_t)^2} \mid x \right]_{f(x_t, \theta, y_t) = \hat{f}_{t-1}(x_t, \theta, y_t)}. \quad (6)$$

While some workarounds exist, for L_1 and other non-smooth losses, the hessian is exactly zero almost everywhere, and the gradient remains binary. This limits the learning capacity of tree-based models and often leads to suboptimal performance [35].

Our approach addresses this challenge through the introduction of an explicit alpha scaling (cost-optimization) block. By searching for an optimal scaling factor after each layer, we directly minimize the final loss without requiring differentiability or continuous gradients. This not only improves the robustness of the ensemble when using non-smooth losses, but also enhances the interaction between different feature groups by allowing their combined effect to be tuned at each stage. In effect, the introduced scheme offers a practical and effective solution to a core limitation of boosting-based ensembles.

Algorithm 2 Hierarchical Ensemble Feature Selection and Prediction.

Require: Dataset $X \in \mathbb{R}^{N \times M}$, target $y \in \mathbb{R}^N$

- 1: **Feature Selection/Scoring:**
- 2: Compute relevance scores for all features (e.g., using AIC, f -regression, etc.)
- 3: Sort features according to their scores
- 4: Decide on the split index p (number of features in dominant subset)
- 5: **Create Feature Subsets:**
- 6: $s_1^{(1)} \leftarrow$ first p features of X ▷ Dominant features
- 7: $s_2^{(1)} \leftarrow$ next q features of X ▷ Non-dominant features
- 8: **(Optional) Scale features and target:**
- 9: $X_1 \leftarrow \text{Scale}(s_1^{(1)})$, $X_2 \leftarrow \text{Scale}(s_2^{(1)})$, $y \leftarrow \text{Scale}(y)$
- 10: **Layer 1: Train base learner on dominant features**
- 11: Fit model f_1 (e.g., LightGBM) on (X_1, y)
- 12: $\hat{y}^{(1)} \leftarrow f_1(X_1)$
- 13: **Alpha computation:**
- 14: Compute α (or α vector) using y and $\hat{y}^{(1)}$
- 15: **Layer 2: Train base learner on non-dominant features + predictions**
- 16: $X_2^+ \leftarrow$ Concatenate X_2 and $\hat{y}^{(1)}$
- 17: Fit model f_2 (e.g., LightGBM) on (X_2^+, α)
- 18: **Prediction:**
- 19: For each test sample x :
- 20: $x_1 \leftarrow$ dominant features of x
- 21: $\hat{y}_1 \leftarrow f_1(x_1)$
- 22: $x_2 \leftarrow$ non-dominant features of x
- 23: $x_2^+ \leftarrow$ Concatenate x_2 and $\hat{y}_1$
- 24: $\hat{\alpha} \leftarrow f_2(x_2^+)$
- 25: $\hat{y} \leftarrow (1 + \hat{\alpha}) \cdot \hat{y}_1$
- 26: **Return** all $\hat{y}$ as final predictions.

3.1.3. Adding new layers

After computing the optimal scaling factor $\alpha_i^{(i)}$ at layer i , we treat this value (or vector) as the new regression target for the next base model, $f^{(i)}$. This model is then trained on the next feature subset $s_i^{(i)}$ together with the current prediction $\hat{y}^{(i-1)}$ (i.e., from the previous layer). The inclusion of $\hat{y}^{(i-1)}$ as an input allows the subsequent model to learn context-dependent corrections conditioned on both the non-dominant features and the prior prediction. This process can be repeated, enabling additional layers if more feature groups are available or finer adjustments are needed. As with previous layers, there is no restriction on the choice of base learner or loss function at this stage.

3.1.4. Final superposition

The final prediction for each sample is obtained by applying the learned scaling factor to the prediction from the previous layer:

$$\hat{y}_t^{(i+1)} = (1 + \hat{\alpha}_t^{(i)}) \hat{y}_t^{(i)} \quad (7)$$

Here, $\hat{\alpha}_t^{(i)}$ is the value predicted by the model in the last layer, and $\hat{y}_t^{(i)}$ is the prior prediction. This formula reflects that the correction is applied multiplicatively (as a relative adjustment), following the structure in Algorithm 2. The procedure can be iterated for as many layers as desired, though we find $K = 2$ is typically sufficient in practice. The result is a prediction that sequentially incorporates information from both dominant and non-dominant feature groups, along with their learned interactions.

Algorithm 2 details the procedure: at each transition from layer $(i - 1)$ to i , $\hat{y}_t^{(i-1)}$ is generated, cost-optimized to produce $\alpha_t^{(i)}$, a new model $f^{(i)}$ is trained on $\{\alpha_t^{(i)}, s_t^{(i)}\}$, and $\hat{y}_t^{(i+1)}$ is computed by scaling. This hierarchical design ensures each feature group has a direct, tunable contribution to the final prediction, combining the strengths of boosting and stacking, but relaxing their technical assumptions on differentiability and feature assignment.

Consequently, our approach is well suited for high-dimensional and non-stationary data, as each layer specializes on a narrower feature subset but corrections and cross-group effects are incorporated through subsequent layers.

In the next section, we evaluate the introduced hierarchical ensemble-based model on synthetic and real-world datasets.

4. Simulations**4.1. Compared models and simulation setup**

We compare our introduced hierarchical ensemble method with several widely used feature selection approaches, following the taxonomy in Section 2.1. For a fair and reproducible comparison, all models use the same base regressor (LightGBM [35]) and are implemented as `scikit-learn` [36] pipelines, enabling easy integration and cross-validation together with the model parameters. LightGBM is chosen for its strong predictive power on tabular and time-series data and efficiency via exclusive feature bundling. A brief explanation of models used in the experiments are as follows:

- **Filter:** Features are sorted by Pearson's correlation coefficient [17] with the target. The top k features (where k is tuned by hyperparameter search) are selected to train a LightGBM model.
- **Wrapper (Forward/Backward):** Features are selected sequentially, based on validation loss: in forward selection, features are added one by one; in backward elimination, features are removed one by one. The number of features is automatically selected by `SequentialFeatureSelection` [36].
- **Embedded:** Features are selected via `SelectFromModel` [36] using LightGBM's feature importances (split gains, etc.). A new LightGBM is then trained on the selected subset.
- **Ensemble:** Two LightGBM models are trained on non-overlapping dominant and non-dominant feature groups, determined via mutual information scores [16]; the split location is tuned as a hyperparameter. Their predictions are combined using a learnable coefficient α_i (see Algorithm 1).
- **Baseline LightGBM:** A single LightGBM trained on the full feature set, providing a reference for models without feature selection.
- **Hierarchical Ensemble (introduced):** Our method, using a two-layer ($K = 2$) hierarchy, explicitly combines dominant and non-dominant features and enables cross-group interaction through a cost optimization block.
- **Hierarchical Ensemble with $K = 3$:** (i.e., three hierarchical layers instead of two).
- **Without Cost Optimization Block:** Hierarchical residual boosting without the adaptive cost optimization step as an ablation study.

Note that whenever it is necessary to determine the number of features—whether for classical algorithms or for our hierarchical feature selection-based algorithms (including the ablation study, HEFS with $K = 2$, and HEFS with $K = 3$)—the value is selected via grid search.

To ensure robust and unbiased comparison, model selection and hyperparameter tuning are performed using 3-fold `TimeSeriesSplit` and `RandomizedSearchCV` [36] over the key hyperparameters summarized in Table 1. After hyperparameter tuning, each experiment is repeated 30 times with different random seeds to obtain reliable statistics. The statistical significance is assessed by a paired one-sided t -test between the MAE of our method and each baseline, with hypotheses $H_0 : \mu_c = \mu_p$, $H_a : \mu_c > \mu_p$, where μ_c and μ_p are the mean errors of the compared and introduced methods, respectively.

4.2. Datasets and feature extraction**4.2.1. Synthetic dataset**

The synthetic time-domain signals are generated with an autoregressive moving average (ARMA) process of order (4, 5), i.e.,

$$y_t = \sum_{i=1}^4 \phi_i y_{(t-i)} + \sum_{j=1}^5 \theta_j \epsilon_{(t-j)} + \epsilon_t, \quad (8)$$

where the autoregressive part is represented by the lagged values up to 4 times, and the moving average is represented by the lagged error

Table 1

Hyperparameter search space for the base LightGBM learners and feature selection algorithms.

Parameter	Search Range	Distribution
<i>Base learner (LightGBM, all layers)</i>		
Learning rate	$[10^{-3}, 0.2]$	(log-uniform)
Number of estimators	[400, 1500]	(int)
Num leaves	[15, 512]	(int)
Max depth	$[-1, 16]$	(int)
Min child samples	[5, 100]	(int)
Min child weight	$[10^{-3}, 10]$	(log-uniform)
Subsample	[0.5, 1.0]	(uniform)
Subsample freq	[0, 8]	(int)
Colsample by tree	[0.5, 1.0]	(uniform)
λ_1 (L1 reg.)	$[10^{-4}, 10]$	(log-uniform)
λ_2 (L2 reg.)	$[10^{-4}, 10]$	(log-uniform)
Max bin	[128, 512]	(int)
Objective	mae, mse	(categorical)
<i>Feature selection algorithms</i>		
Number of selected features (k)	[5, 30]	(int)
Threshold for SelectFromModel	[0.5, 1.0]	(uniform)
<i>Hierarchical ensemble structure</i>		
β (cost optimization)	[0.05, 1.0]	(uniform)
Feature split location(s)	All $(p, q, (r))$ with $p + q + (r) = \text{num_feat}$	

terms up to 5 times. The ϕ and θ variables control the strength of the lags [37]. In our setting, $\phi = [0.4, 0.3, 0.2, 0.1]$, and $\theta = [0.65, 0.35, 0.3, -0.15, -0.3]$. We set up weights for five lags so that the correlation is enough to prompt a seasonality and trend in the data. We tested the non-stationarity using the Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test [38], and the test results showing non-stationarity can be seen in detail in Table 2. Then, the series is transformed with a min-max scaler. We generate 300 synthetic signal samples to simulate the difficulty of base models to capture trends in time-domain signal. As a result, the synthetic data is equipped with non-stationarity, trend, and seasonality.

In our synthetic experiments, we explicitly constructed both dominant and non-dominant feature sets to realistically represent the hierarchical feature grouping used in our method. The dominant features consist of lagged values of the target signal, which are well known to dominate predictive performance in time series modeling [5]. For the non-dominant set, we generated a collection of side-information features by applying a separate feature extraction process, designed to be informative yet distinct from the main target dynamics. Specifically, we generated a block of side information using the `make_classification` utility from `sklearn` [36], following the NIPS 2003 variable selection benchmark [39]. We set both the total number of features and the number of informative features to 26 ($n_{\text{features}} = n_{\text{informative}} = 26$), with no redundant or repeated variables. Thus, all side-information features are fully informative with respect to the binary class labels but are not direct functions of the regression target. Then, the y_t values obtained from (8) that have indexes in the corresponding label 1 in y_t^b are multiplied by 1.4 to upscale, while others are multiplied by 0.6 to downscale. This design allows us to control the amount and structure of informative features while keeping the problem realistic and challenging for feature selection methods. Therefore, we guarantee dependence on the generated domain knowledge series, which can provide a substantial amount of pattern. As the final step of generation, we added a Gaussian normal noise of $N(\mu = 0, \sigma = 0.5)$.³

³ The noise level ($\sigma = 0.5$) is set empirically to produce a moderate signal-to-noise ratio and is held fixed for all algorithms. Since the main purpose of this synthetic experiment is to test the utilization of non-dominant features in a controlled setting, the specific value of the noise variance is of secondary importance. This ensures a consistent and challenging test environment.

Table 2

Stationarity analysis and dimensionality of the datasets. For each dataset we report the number of features, number of samples, and the combined KPSS p -value. Individual KPSS test p -values were first corrected for multiple testing with the Benjamini–Yekutieli false-discovery-rate procedure [42]. The adjusted p -values were then aggregated across all series using Fisher’s method to yield the “Combined KPSS p -values” statistic shown [43]. Smaller combined KPSS p -values provide sufficient evidence to reject the null hypothesis that each series is (level/trend) stationary, indicating that the data are non-stationary.

Dataset	Num. features	Num. samples	Combined KPSS p -values
Synthetic	36	300	5.4×10^{-11}
M4 Yearly	33	400	4.4×10^{-3}
Weather	40	300	1.0×10^{-2}

4.2.2. Real datasets

We conduct experiments on two real-world datasets: the M4 competition dataset [40] and the Weather dataset [41]. Both benchmarks are widely used in time series forecasting research and present challenges of non-stationarity and high dimensionality (see Table 2).

M4 competition dataset The yearly M4 competition dataset [40] contains 23000 business-related time series from finance and economics, making it a standard benchmark for forecasting methods. For each experimental run, the dataset is randomly sub-sampled to ensure high dimensionality. Since the raw dataset lacks explicit date-time indices, we generate them externally to enable extraction of calendar-based features.

Feature construction follows a two-group approach: dominant features are engineered from the 2nd, 4th, 6th, and 8th lags of the target, along with rolling window statistics (mean and standard deviation) at those lags. These features are assigned to $s_t^{(1)}$ to ensure comparability with the synthetic setting and to guarantee the existence of a dominant group. Calendar features—including hour, day, week, month, and quarter, all encoded as sine and cosine transforms—are included as non-dominant features ($s_t^{(2)}$). All features are min-max scaled. The non-stationarity of the M4 dataset is checked using KPSS test [38] as shown in Table 2.

Weather dataset The Weather dataset [41] consists of meteorological measurements, including variables such as temperature, humidity, wind speed, and atmospheric pressure. For our experiments, we set the prediction target as temperature ($y = T$ (degC)), and treated the remaining measurements as features. In addition, we engineered lagged features and calculated rolling means and standard deviations over several relevant time windows (e.g., 1 hour, 1 day, 1 week, and 1 month), to capture both short- and long-term temporal dynamics.

Unlike the synthetic and M4 datasets, the separation between dominant and non-dominant features here was not determined by domain expertise. Instead, we adopted a classical feature selection approach: all features were first ranked by their mutual information [16] with the target variable, and the set was split into dominant and non-dominant groups based on a hyperparameter-tuned split location (see split locations in Table 1). This approach demonstrates the flexibility of our hierarchical method, as discussed in Section 3, allowing for alternative and data-driven procedures for feature grouping.

Both experiments represent real-world forecasting problems where the number of features is large and the underlying time series exhibit strong non-stationary patterns, as shown in Table 2, validating the need for robust and adaptive feature selection strategies.

5. Experimental results

Test and train mean absolute errors (MAE) for each model and dataset are summarized in Tables 3 and 4, respectively. Table 5 gives statistical significance results based on paired t -tests, and Table 6 reports the average training time for each method. The results indicate that the

Table 3

Test MSE (lower is better).

Model	Synthetic	M4	Weather
Filter	1.023	0.105	0.0123
Forward Wrapper	1.004	0.106	0.0111
Backward Wrapper	0.980	0.107	0.0110
Embedded	0.978	0.109	0.0135
Ensemble	0.965	0.111	0.0134
Baseline LightGBM	0.980	0.105	0.0120
HEFS ($K=2$)	0.956	0.103	0.0110
HEFS ($K=3$)	0.977	0.111	0.0153
HEFS (Ablation)	0.962	0.106	0.0122

Table 4

Train MSE (lower is better).

Model	Synthetic	M4	Weather
Filter	0.900	0.0739	0.00382
Forward Wrapper	0.697	0.0379	0.00168
Backward Wrapper	0.858	0.0746	0.00214
Embedded	0.961	0.0210	0.00721
Ensemble	0.655	0.0281	0.00530
Baseline LightGBM	0.941	0.0388	0.00447
HEFS ($K=2$)	0.725	0.0235	0.00630
HEFS ($K=3$)	0.644	0.0673	0.00451
HEFS (Ablation)	0.468	0.0202	0.00570

Table 5Paired t -test results comparing HEFS ($K=2$) against baseline models on each data set.

Model	Synthetic		M4		Weather	
	t -score	p -value	t -score	p -value	t -score	p -value
Filter	6.330	0.0000	2.025	0.0286	4.740	0.0000
Forward Wrapper	5.149	0.0000	1.004	0.1640	0.254	0.4010
Backward Wrapper	3.224	0.0016	1.522	0.0722	0.042	0.4830
Embedded	3.049	0.0024	3.199	0.0024	8.609	0.0000
Ensemble	1.107	0.1390	3.988	0.0004	7.387	0.0000
Baseline LightGBM	3.232	0.0015	1.036	0.1570	3.577	0.0006
HEFS ($K=3$)	3.584	0.0000	4.497	0.0001	17.16	0.0000
HEFS (Ablation)	0.967	0.1710	1.588	0.0644	3.419	0.0009

introduced HEFS approach achieves the lowest test error on both the synthetic and real datasets, outperforming all other models. This highlights the robustness and adaptability of HEFS, especially in complex, high-dimensional, and non-stationary environments.

On the Weather dataset, while wrapper-based methods (both forward and backward) attain test accuracies very close to that of HEFS. However, Table 6 shows that the training times for wrapper-based methods are substantially higher. This makes HEFS a much more attractive choice in practical applications where computational resources or time are limited.

Ablation experiments reveal the contribution of the cost optimization block: models with this component achieve better test accuracy without a substantial increase in training time. Considering the fact the lowest training errors for two of the datasets are achieved by ablation method, we can conclude that the cost optimization block helps control overfitting and improves generalization, likely by promoting a more effective utilization of the feature space.

Similar findings are observed when increasing the hierarchy depth. The HEFS ($K=3$) variant, although it includes an additional base learner and cost optimization stage, achieves a reduction in training time in some cases (see Table 6) when it is compared to the HEFS ($K=2$). This result suggests that deeper hierarchical structures with smaller base learners may offer further improvements in terms of overfitting mitigation and should be explored in future research.

Table 6

Training times (in seconds). All experiments were run on a 14-inch MacBook Pro with an Apple M3 chip (8-core CPU, 10-core GPU) and 16 GB of unified memory. Training times correspond to the `mean_fit_time` reported by each model's `RandomizedSearchCV` object.

Model	Training time (s)		
	Synthetic	M4	Weather
Filter	0.316	1.763	6.048
Forward Wrapper	153.2	71.38	128.9
Backward Wrapper	265.9	135.5	99.60
Embedded	0.531	11.89	2.572
Ensemble	11.74	13.26	3.533
Baseline LightGBM	0.556	6.309	0.591
HEFS ($K=2$)	2.478	10.56	2.238
HEFS ($K=3$)	3.779	5.453	1.263
HEFS (Ablation)	4.231	11.16	1.407

Overall, these results show that the hierarchical, cost-optimized ensemble approach not only achieves superior predictive accuracy, but also offers greater computational efficiency and robustness to overfitting, making it a promising tool for high-dimensional time series modeling.

5.1. Limitations

While our hierarchical ensemble framework demonstrates improved predictive performance in high-dimensional, non-stationary settings, there are limitations to consider. First, when ample training data are available, most modern machine learning models already tend to leverage all informative features and do not overfit solely to a small subset of dominant features; in such cases, the incremental benefit of our hierarchical approach could diminish. Second, our method relies on the existence of clear dominant feature groups—if such structure is absent and all features contribute equally, the hierarchical framework may not yield a substantial advantage over standard approaches. Finally, feature grouping in this study was performed manually, using simple domain heuristics or single information criteria. Developing automated or more sophisticated feature grouping strategies remains an open question for future work. Despite these limitations, our framework provides a flexible basis for hierarchical learning in complex or evolving data environments. Additionally, as the number of layers is further increased (i.e., $K > 2$), our hierarchical structure could be adapted to use much simpler base models in each layer, similar to the weak learners in classical gradient boosting. This would allow the ensemble to incrementally refine its predictions in a more gradual and smooth manner, potentially decreasing the gap between hierarchical stacking and boosting frameworks.

6. Conclusion

In this work, we introduce a hierarchical ensemble-based feature selection method for high-dimensional, non-stationary time series prediction. Unlike traditional methods that typically focus on a single dominant feature subset, our approach uses a multi-layer structure where dominant features are processed first, followed by sequential refinement with non-dominant features. A dedicated cost-optimization block between layers recalibrates predictions by minimizing the final loss, capturing complex inter-feature interactions that classical stacking-based ensembles might overlook. Experimental results confirm that our hierarchical approach provides superior predictive accuracy and robustness against overfitting compared to standard methods. However, the incremental advantage may diminish when sufficient data allow traditional models to effectively utilize all features, or if the dataset lacks clear dominant feature groups. Future research opportunities include developing automated or advanced feature grouping strategies and further extending the hierarchy depth with simpler base models. Overall, our flexible

framework represents a promising approach for hierarchical learning in complex and evolving data environments. To support reproducibility and further research, we openly share our code.

CRedit authorship contribution statement

- **Hüseyin Karaca:** Investigation; Writing - original draft; Writing - review & editing. (Designed and ran the experiments, drafted the initial manuscript, and led subsequent revisions.)
- **Aysin Tumay:** Investigation; Visualization; Writing - original draft. (Conducted experiments, prepared all figures, and contributed to the first manuscript draft.)
- **Ali Taha Koç:** Conceptualization; Supervision; Writing - review & editing. (Provided conceptual guidance, supervised the research process, and critically revised the manuscript.)
- **Serdar Kozat:** Supervision; Writing - review & editing. (Oversaw the study, engaged in technical discussions, and participated in manuscript revision.)

All authors reviewed and approved the final version of the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data is publicly available.

References

- [1] V. Bolón-Canedo, A. Alonso-Betanzos, Ensembles for feature selection: a review and future trends, *Inf. Fusion* 52 (2019) 1–12, <https://doi.org/10.1016/j.inffus.2018.11.008>.
- [2] M. Dehghan, J.H. de, M. Goulart, A.L. de Almeida, Low-rank multilinear filtering, *Digit. Signal Process.* 153 (2024) 104646, <https://doi.org/10.1016/j.dsp.2024.104646>.
- [3] J.H. Friedman, On bias, variance, 0/1—loss, and the curse-of-dimensionality, *Data Min. Knowl. Discov.* 1 (1997) 55–77, <https://doi.org/10.1023/A:1009778005914>.
- [4] Z. Yang, Y. Chen, Cross-dimensional global interactive transformer for traffic forecasting, *Digit. Signal Process.* 159 (2025) 104974, <https://doi.org/10.1016/j.dsp.2024.104974>.
- [5] M. Du, Improving LSTM neural networks for better short-term wind power predictions, in: 2019 IEEE 2nd International Conference on Renewable Energy and Power Engineering (REPE), 2019, pp. 105–109.
- [6] J.R. Quinlan, Induction of decision trees, *Mach. Learn.* 1 (1986) 81–106, <https://doi.org/10.1007/BF00116251>.
- [7] R. Kohavi, G.H. John, Wrappers for feature subset selection, *Artif. Intell.* 97 (1–2) (1997) 273–324, [https://doi.org/10.1016/S0004-3702\(97\)00043-X](https://doi.org/10.1016/S0004-3702(97)00043-X).
- [8] R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. Ser. B, Methodol.* 58 (1) (1996) 267–288, <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- [9] L. Breiman, Random forests, *Mach. Learn.* 45 (2001) 5–32, <https://doi.org/10.1023/A:1010933404324>.
- [10] H. Akaike, A new look at the statistical model identification, *IEEE Trans. Autom. Control* 19 (6) (1974) 716–723, <https://doi.org/10.1109/TAC.1974.1100705>.
- [11] G. Schwarz, Estimating the dimension of a model, *Ann. Stat.* (1978) 461–464.
- [12] L. Martino, R. San Millán-Castillo, E. Morgado, Spectral information criterion for automatic elbow detection, *Expert Syst. Appl.* 231 (2023) 120705, <https://doi.org/10.1016/j.eswa.2023.120705>.
- [13] E. Morgado, L. Martino, R.S. Millán-Castillo, Universal and automatic elbow detection for learning the effective number of components in model selection problems, *Digit. Signal Process.* 140 (2023) 104103, <https://doi.org/10.1016/j.dsp.2023.104103>.
- [14] A. Hashemi, M.B. Dowlatshahi, H. Nezamabadi-pour, Mfs-mcdm: multi-label feature selection using multi-criteria decision making, *Knowl.-Based Syst.* 206 (2020) 106365, <https://doi.org/10.1016/j.knosys.2020.106365>.
- [15] G. Ridgeway, Generalized boosted models: a guide to the gbm package, Update 1 (1) (2007) 2007.
- [16] A. Kraskov, H. Stögbauer, P. Grassberger, Estimating mutual information, *Phys. Rev. E* 69 (6) (Jun. 2004), <https://doi.org/10.1103/physreve.69.066138>.
- [17] K. Pearson, On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling, *Philos. Mag.* 50 (302) (1900) 157–175, <https://doi.org/10.1080/14786440009463897>.
- [18] R. Fano, *Transmission of Information: A Statistical Theory of Communications*, MIT Press, Cambridge, Mass, 1961.
- [19] M.A. Efronson, Multiple regression analysis, *Math. Methods Digit. Comput.* 1 (1960) 191–203.
- [20] R.S. Millán-Castillo, L. Martino, E. Morgado, F. Llorente, An exhaustive variable selection study for linear models of soundscape emotions: rankings and Gibbs analysis, *IEEE/ACM Trans. Audio Speech Lang. Process.* 30 (2022) 2460–2474, <https://doi.org/10.1109/TASLP.2022.3192664>.
- [21] M.E. Aydin, S.S. Kozat, A hybrid framework for sequential data prediction with end-to-end optimization, *Digit. Signal Process.* 129 (2022) 103687, <https://doi.org/10.1016/j.dsp.2022.103687>.
- [22] Y. Saeyns, I. Inza, P. Larrañaga, A review of feature selection techniques in bioinformatics, *Bioinformatics* 23 (19) (2007) 2507–2517, <https://doi.org/10.1093/bioinformatics/btm344>.
- [23] Z. Ma, Y. Wei, Y. Huang, J. Wang, Unsupervised feature selection based on minimum-redundant subspace learning with self-weighted adaptive graph, *Digit. Signal Process.* 155 (2024) 104738, <https://doi.org/10.1016/j.dsp.2024.104738>.
- [24] S. Das, Filters, wrappers and a boosting-based hybrid for feature selection, in: *Proceedings of the International Conference on Machine Learning, USA, 2001*.
- [25] M. Moran, G. Gordon, Deep curious feature selection: a recurrent, intrinsic-reward reinforcement learning approach to feature selection, *IEEE Trans. Artif. Intell.* 5 (3) (2024) 1174–1184, <https://doi.org/10.1109/TAI.2023.3282564>.
- [26] W. Sheng, X. Li, Multi-task learning for gait-based identity recognition and emotion recognition using attention enhanced temporal graph convolutional network, *Pattern Recognit.* 114 (2021) 107868, <https://doi.org/10.1016/j.patcog.2021.107868>.
- [27] W. Sheng, X. Li, Siamese denoising autoencoders for joints trajectories reconstruction and robust gait recognition, *Neurocomputing* 395 (2020) 86–94, <https://doi.org/10.1016/j.neucom.2020.01.098>.
- [28] A. Jenul, S. Schrunner, J. Pilz, O. Tomic, A user-guided Bayesian framework for ensemble feature selection in life science applications (ubayfs), *Mach. Learn.* (2022), <https://doi.org/10.1007/s10994-022-06221-9>.
- [29] Nist digital library of mathematical functions, equation (5.19.4), Release 1.2.4, <https://dlmf.nist.gov/5.19#E4>, 2025. (Accessed 28 May 2025).
- [30] R.E. Bellman, *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957.
- [31] F.S. Duarte, R.A. Rios, E.R. Hruschka, R.F. de Mello, Decomposing time series into deterministic and stochastic influences: a survey, *Digit. Signal Process.* 95 (2019) 102582, <https://doi.org/10.1016/j.dsp.2019.102582>.
- [32] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Springer, 2009.
- [33] I. Guyon, A. Elisseeff, An introduction to variable and feature selection, *J. Mach. Learn. Res.* 3 (2003) 1157–1182.
- [34] J.H. Friedman, Greedy function approximation: a gradient boosting machine, *Ann. Stat.* 29 (5) (2001) 1189–1232, <http://www.jstor.org/stable/2699986>.
- [35] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: a highly efficient gradient boosting decision tree, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 3149–3157.
- [36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: machine learning in python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [37] G.E.P. Box, G.M. Jenkins, *Time Series Analysis: Forecasting and Control*, Holden-Day, San Francisco, 1970.
- [38] D. Kwiatkowski, P.C. Phillips, P. Schmidt, Y. Shin, Testing the null hypothesis of stationarity against the alternative of a unit root: how sure are we that economic time series have a unit root?, *J. Econom.* 54 (1) (1992) 159–178, [https://doi.org/10.1016/0304-4076\(92\)90104-Y](https://doi.org/10.1016/0304-4076(92)90104-Y).
- [39] I. Guyon, Design of experiments of the nips 2003 variable selection benchmark, in: *NIPS 2003 Workshop on Feature Extraction and Feature Selection*, vol. 253, 2003, p. 40.
- [40] S. Makridakis, E. Spiliotis, V. Assimakopoulos, The m4 competition: results, findings, conclusion and way forward, *Int. J. Forecast.* 34 (4) (2018) 802–808, <https://doi.org/10.1016/j.ijforecast.2018.06.001>.
- [41] W. Haixu, X. Jiehui, J. Wang, L. Mingsheng, *Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting*, 2022.
- [42] Y. Benjamini, D. Yekutieli, The control of the false discovery rate in multiple testing under dependency, *Ann. Stat.* 29 (4) (2001) 1165–1188, <https://doi.org/10.1214/aos/1013699998>.
- [43] R.A. Fisher, *Statistical Methods for Research Workers*, Oliver and Boyd, Edinburgh, 1925.